Model 3: Ordering plan

Nov 7, 2026 · @alan

What it does and why it matters most

Model 3 decides how much of each material to order each week and how to split the order across supplier countries. It fills `order_units` and the 6 share columns (`share_KES` to `share_AUR`), and those columns directly control 70% of the score.

Score part	Share of total	What drives it
F: financial	40%	Total cost of our orders vs the naive policy and a perfect-hindsight plan
R: fill rate in shock weeks	21%	How much demand we meet during shocks and 4 recovery weeks after
R: diversification	9%	How evenly orders are spread across each material's 3 suppliers

It's an optimization model (a linear program, or LP), not a trained model. It uses Model 1's shock probabilities to decide how much to hedge and Model 2's landed costs to decide who to buy from and when.

How the simulator scores orders

The competition's simulator plays our orders out against what really happened. Each window (weeks 209 to 221 and 222 to 234) is simulated on its own, and only the private window (222 to 234) counts for final standings.

1. **Start:** 5 weeks of demand in stock, nothing in transit.
2. **Each week's order** is split by our shares. Each country's slice is filled only up to that country's capacity that week; units it can't fill are cancelled and **not charged**.
3. **Filled units** are paid at that week's landed cost and arrive after that week's lead time.
4. **Each week,** arrivals go into stock, then Halcyon uses up to that week's demand.
5. **Costs:** holding stock costs 2% of the average landed cost per unit per week; each unit of unmet demand costs 5× the average landed cost.

6. **End of window:** units in stock or still in transit are sold back at 70% of the average landed cost.

What this means:

- **Running out costs far more than overbuying.** A stockout costs 5×; an unused unit loses about 30% plus holding. Keeping a buffer is worth it, more so when risk is high.
- **Demand is known** for every week (demand.csv), so all uncertainty is on the supply side.
- **Ordering extra is cheap insurance** when capacity might drop, since unfilled units cost nothing.
- **Stop ordering near the end:** units still in transit at week 234 lose 30%.

Quick version: one forecast, plus a safety buffer

The quick version is our original LP solved for one material and one 13-week window at a time, assuming the forecasts are right, with two fixes. It's enough for a first full submission.

What it minimizes: purchase cost + holding cost + stockout penalty − end-of-window sell-back.

Decisions: units ordered from each supplier each week, stock at the end of each week, and unmet demand each week.

Rules (constraints):

- Stock this week = stock last week + arrivals − demand + unmet demand.
- Each supplier gets at least a minimum share of each week's order (s_min, for example 0.2).

Fix 1: separate ordered from filled units. In the original model, a supplier with zero capacity forced the whole week's order to zero, because the minimum share applied to what was filled. Split it into:

- **o** = units ordered (what we submit; the minimum share applies here)
- **f** = units actually filled, with f ⩽ o and f ⩽ capacity. We pay for f and receive f.

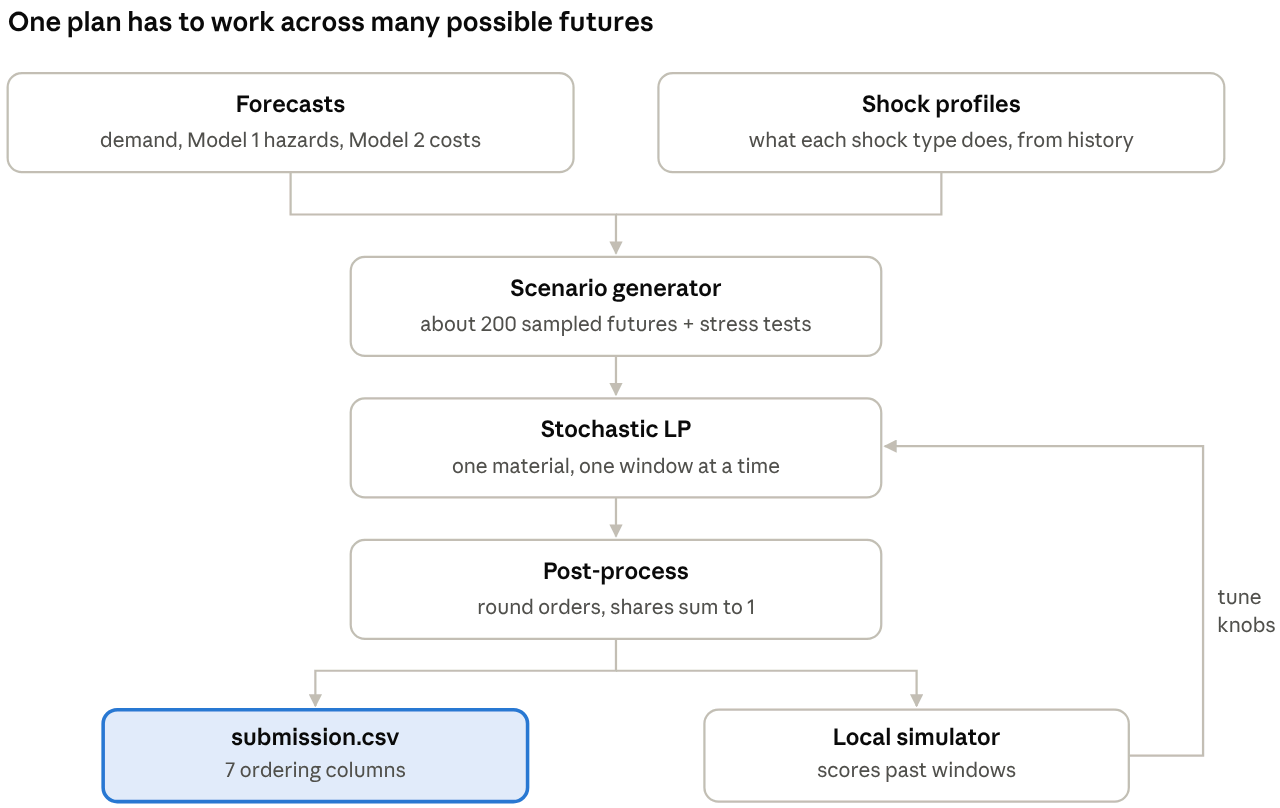
Fix 2: add a safety buffer. With perfect forecasts the LP holds no spare stock, so a real shock hits with the full 5× penalty. As a stand-in for proper scenarios, require:

```
I_t \ge k \cdot p_t \cdot d_t
```

where I_t is stock at the end of week t, p_t is Model 1's disruption probability, d_t is demand, and k is about 2 to 4 weeks. The buffer grows when risk is higher.

Full version: planning for many possible futures

Our submission fixes every order for all 13 weeks before anything happens; we can't react once a shock hits. So the full version picks **one set of orders that works well on average across many possible futures**, some calm and some with shocks. This is called a two-stage stochastic LP.



full ordering pipeline · with tuning loop

How it works in plain terms:

- **Stage 1 (shared):** the orders o, chosen once. This is what we submit.
- **Stage 2 (one copy per future s):** how those orders play out in that future: units filled, stock, and shortfall.
- **Objective:** minimize the weighted average cost across all futures.

Because some futures contain shocks, the LP buys extra stock and spreads orders across suppliers only as far as it pays off. Safety stock comes out of the math instead of a hand-set rule.

Each future changes three things for each supplier and week:

Multiplier	Meaning	Example
κ (capacity)	1 = normal, 0 = nothing available	Export ban: κ = 0
δ (extra lead weeks)	Orders take longer to arrive	Strait closure: δ = +3
μ (cost)	Price multiplier	Tariff hike: μ = 1.2

The model:

```
\min \sum_s w_s \Big[ \sum_{c,t} p^s_{c,t} f^s_{c,t} + \sum_t h\, I^s_t +
\sum_t \tilde\pi\, u^s_t - r \big( I^s_{13} + \text{in-transit}^s \big) \Big]
```

- $I^s_t = I^s_{t-1}$ + arrivals − d_t + u^s_t (stock balance in each future)
- $f^s \leqslant o$ and $f^s \leqslant \text{capacity}^s$ (filled units capped by order and capacity)
- $f^s \geqslant o$ − (normal capacity − capacity^s), with $o \leqslant$ normal capacity (stops the LP from filling less than the simulator would)
- o for each supplier ⩾ s_min × total order in week t (minimum share)

Internal penalty. Inside the LP the stockout penalty is λ × average cost with λ between 5 and 8. Going above the real 5 trades a little cost for extra fill rate during shocks, which the resilience score rewards.

Size: about 14,000 variables per LP; `scipy.optimize.linprog(method="highs")` solves each in under a second, so all 20 LPs (10 materials × 2 windows) take seconds.

Scenario generator

The LP is only as good as the futures we give it, so this is where most of the remaining effort should go. Generate about 200 sampled futures, using **the same draws for all 10 materials** so a strait closure hits every material from those countries at once.

In each sampled future:

1. **Lanes:** for each shipping lane, draw whether a closure or strike starts, its start week (from Model 1′s lane hazards) and its duration (from Model 1′s duration curve). It hits every country on the lane.
2. **Countries:** draw bans, sanctions and tariff shocks the same way, from the country hazards.

3. **Materials:** draw resource shortages, which hit all of that material's suppliers.
4. **Ongoing shocks:** anything active at the forecast point carries into every future, with a sampled remaining duration.
5. **Effects:** apply κ, δ and μ for the shock type from the shock profiles, with some random noise.
6. **No-warning reversals:** add a small baseline chance in every future.

Shock profiles come from the event study: in supply_history during past shocks, measure how far capacity dropped, how much lead time grew, how much cost rose, and how long each type lasted.

Stress scenarios. Add about 5 to 10 futures, with total weight ω of 0.1 to 0.2, that are bigger than anything in training. This is our defense against the big shock expected in the test period.

- Merrow Strait closed for 8 weeks, starting in week 3
- Each material's cheapest supplier banned for the whole window
- Capacity cut 50% across all suppliers of a material

Post-processing and submission checks

Most rejected submissions fail here, so turn the LP's answer into the CSV carefully and validate before uploading.

- **order_units** = the week's total order across suppliers, rounded to a whole number, 0 or more.
- **Shares** = each supplier's order ÷ the total. Exactly 0 for any country that doesn't sell the material (check suppliers.csv), and renormalized so each row sums to exactly 1.
- **Zero orders:** if a week's total is 0, split shares evenly across the material's 3 suppliers.
- **Both windows:** solve weeks 209 to 221 and 222 to 234 separately, each starting from 5 weeks of stock.

Before every upload, check:

- [ ] Exactly 260 rows (10 materials × 26 weeks), ids like `w222_lithium`
- [ ] No missing, duplicate or blank values
- [ ] disruption_prob between 0 and 1; price_forecast above 0
- [ ] Shares sum to 1 on every row, 0 for countries that don't sell the material

Local simulator and tuning

Build the local simulator first. Without it we're tuning blind, and Kaggle only allows 5 submissions a day.

The simulator follows the competition rules exactly (see "How the simulator scores orders") and runs on past windows using the real values in supply_history. From it we can compute, locally:

- **F (financial):** our cost vs the naive policy's cost (from sample_submission) and vs the hindsight cost. The hindsight cost is the same LP solved with the actual past values as its only future.
- **Fill-rate skill** in shock weeks plus 4 recovery weeks.
- **Diversification:** 1 − HHI of our shares, rescaled so single sourcing is 0 and an even split is 1.

Knobs to tune:

Knob	What it controls	Values to try
s_min	Minimum share per supplier	0.1, 0.2, 0.33
λ	Internal stockout penalty (× average cost)	5, 6, 8
ω	Total weight of stress scenarios	0.1, 0.2, 0.3
Probability multiplier	Scales Model 1's probabilities up	1, 1.5

Test windows: starting at weeks 157, 170, 183 and 196 (add 209 after Phase 2), plus one window with an injected big shock (for example, Merrow Strait closed 8 weeks from week 5).

Pick the settings that do best in the **worst** window, not the best average: with a big shock coming, robustness beats squeezing out the last bit of cost.

Check one detail early: whether "5 weeks of demand in stock" means the sum of the first 5 weeks or 5 × average demand. Run the naive policy through both versions and see which matches its expected behavior.

Weak spots, final submissions and build order

The structure is right; the gains from here come from better scenarios, honest stress testing, and two genuinely different final submissions, not from more complex math.

Known weak spots:

- **Scenarios are the biggest risk.** If they come only from calm training history, the LP under-hedges the big shock however good the math is.
- **Small approximations:** the filled-units bound is slightly optimistic for partial capacity cuts, and the average cost and starting stock are estimates. The simulator loop catches these.
- **Ordering above normal capacity is blocked.** If supply_history shows capacity sometimes above normal, we're missing free insurance, since unfilled units cost nothing. A 2-minute check.
- **Diversification is a floor (s_min), not a tradeoff,** because the real metric is quadratic. Tuning s_min covers most of the gap.

Two final submissions. Each score part is clipped to 0 to 1, so a cost worse than naive scores 0, not negative, and Kaggle keeps the better of our two picks. That favors making them genuinely different:

- **A, moderate:** the robust settings from tuning.
- **B, aggressive:** higher ω and probability multiplier, λ = 8, s_min = 0.33.

Build order:

- [ ] Local simulator + naive baseline score (1.5 hr)
- [ ] Quick LP with both fixes → first full submission (1.5 hr)
- [ ] Shock profiles from the event study (1 hr)
- [ ] Scenario generator + stochastic LP (2 to 3 hr)
- [ ] Tuning grid + injected-shock test (1.5 hr)
- [ ] Final A and B submissions, selected by Sunday 12 PM Central (30 min)